USER DATAGRAM PROTOCOL

CIS*3210
Fall 2026

Sajid Marhon

Based on notes for Computer Networking: A Top-Down Approach, 9th ed., by J. Kurose and K. Ross

Learning Outcomes



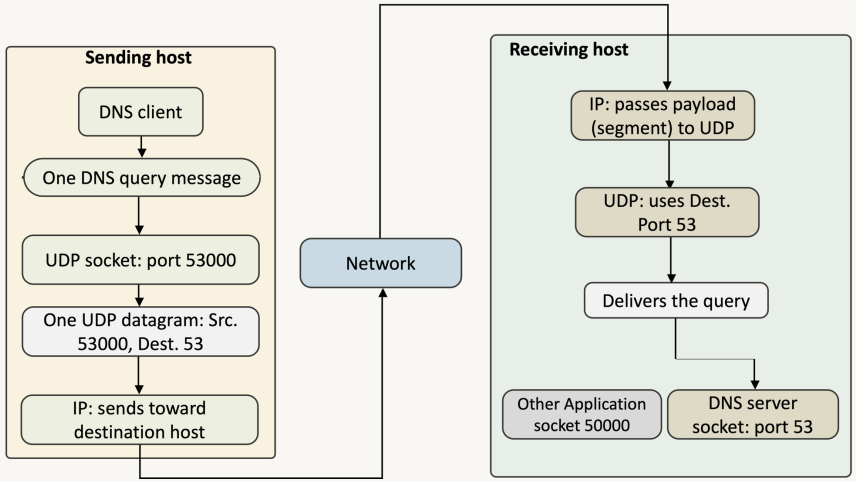
By the end of this lesson, you will be able to:

- Describe UDP's services and header fields
- Explain how message size affects transmission efficiency
- Calculate and verify UDP checksums and recognize their limitations

The User Datagram Protocol



- TCP/IP suite contains two transport protocols
  - User Datagram Protocol (UDP)
  - Transmission Control Protocol (TCP)
- They differ in the service they offer to applications
- UDP is less complex and easy to understand
- UDP can be characterized as:
  - End-to-end
    - UDP can distinguish among multiple applications running on a given computer
  - Connectionless
  - Message-oriented
    - UDP sends and receives individual messages





Discussion Question:

A developer chooses UDP only because it is less complex than TCP. Is that enough to decide whether UDP is suitable for an application?

No. The developer must also consider the service the application needs. A simpler transport protocol is useful only if its delivery service meets the application's needs, or the application supplies any additional functions it requires.

The User Datagram Protocol



- Best-effort Delivery
  - It offers applications the same best-effort delivery semantics as IP
- Arbitrary Interaction
  - UDP allows an application
    - to send to many other applications
    - receive from many other applications
    - or communicate with exactly one other application
- Operating System Independent
  - UDP provides a means of identifying application programs
  - It does not depend on identifiers used by the local OS
- UDP is sometimes characterized as a thin protocol layer

The Connectionless Paradigm



- UDP uses a connectionless communication paradigm
  - means that an application using UDP does not need to pre-establish communication before sending data
  - an application can generate and send data at any time
  - UDP allows an application to delay an arbitrarily long time between the transmission of two messages
- UDP does not connection maintain state
- UDP does not use control messages
  - communication consists only of the data messages themselves
- UDP has extremely low overhead

Message-Oriented Interface



- UDP offers applications a message-oriented interface
- Each time an application requests that UDP send data
  - UDP places the data in a single message for transmission
  - UDP does not divide an application message into multiple UDP datagrams
  - UDP does not combine messages for delivery
- Message-oriented interface has several consequences:
  - On the positive side
    - applications preserve data boundaries
      - each message will be exactly the same as was transmitted
  - On the negative side
    - each UDP message must fit into a single IP datagram

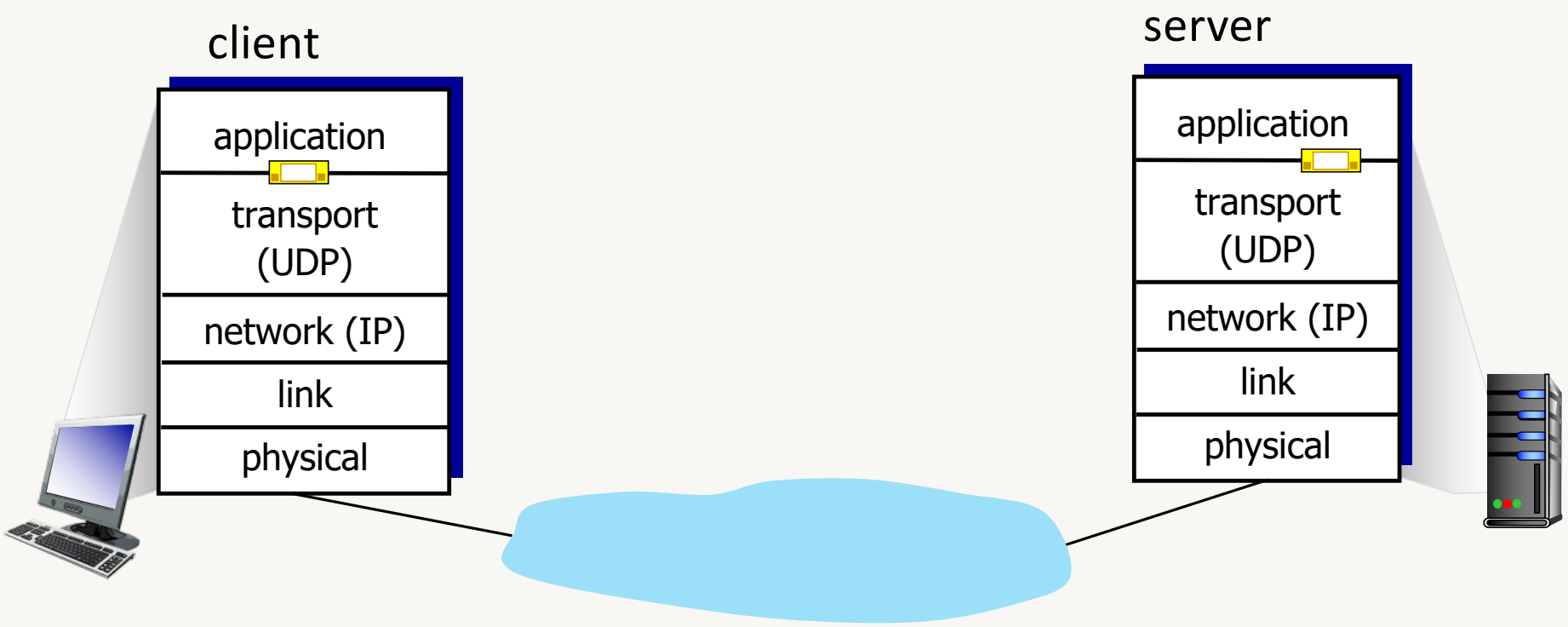
Message-Oriented Interface



- IP datagram size limits the maximum UDP message size
- Very small messages have high header overhead relative to their data
- Very large messages may produce IP datagrams that exceed the network maximum transmission unit (MTU) and require fragmentation, reducing efficiency
- For better efficiency, use larger messages while keeping the complete IP datagram, including headers, within the path MTU

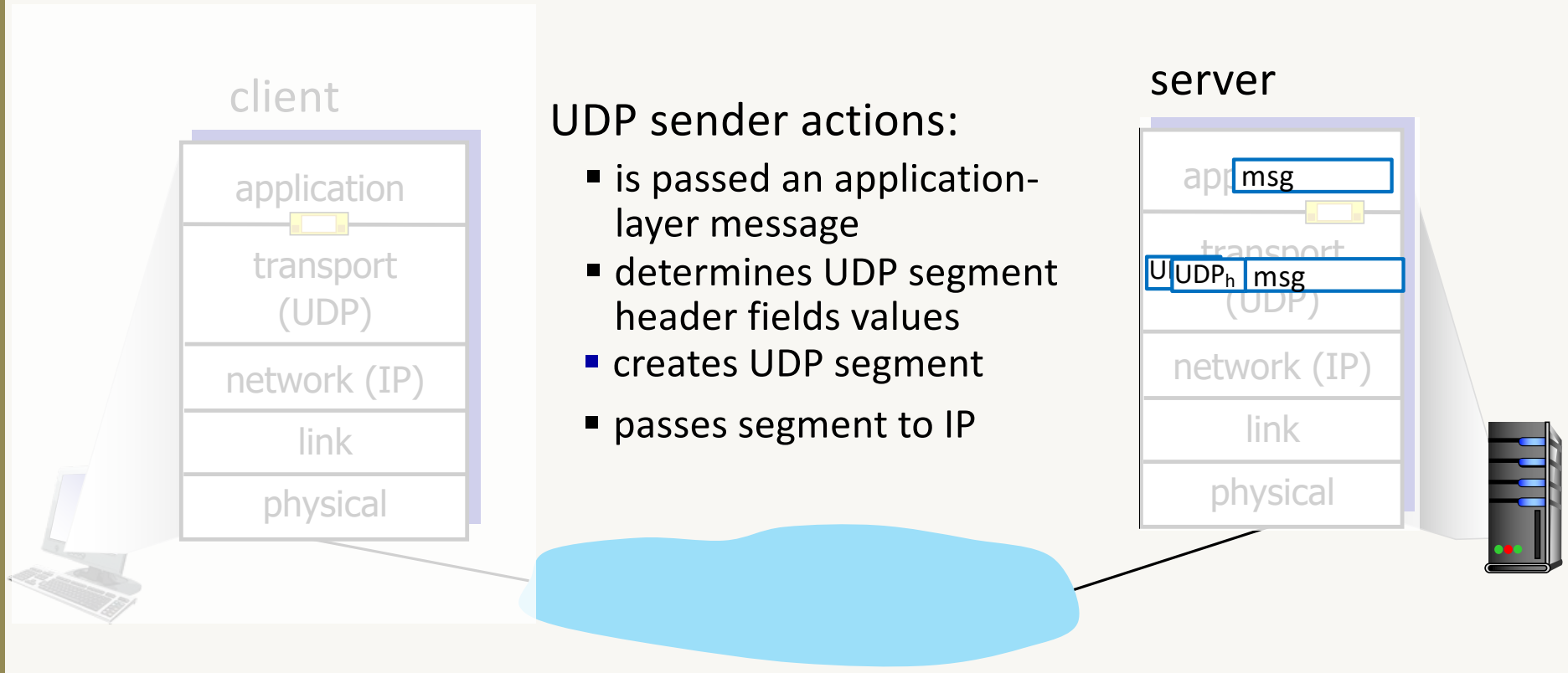
UDP: Transport Layer Actions





UDP: Transport Layer Actions



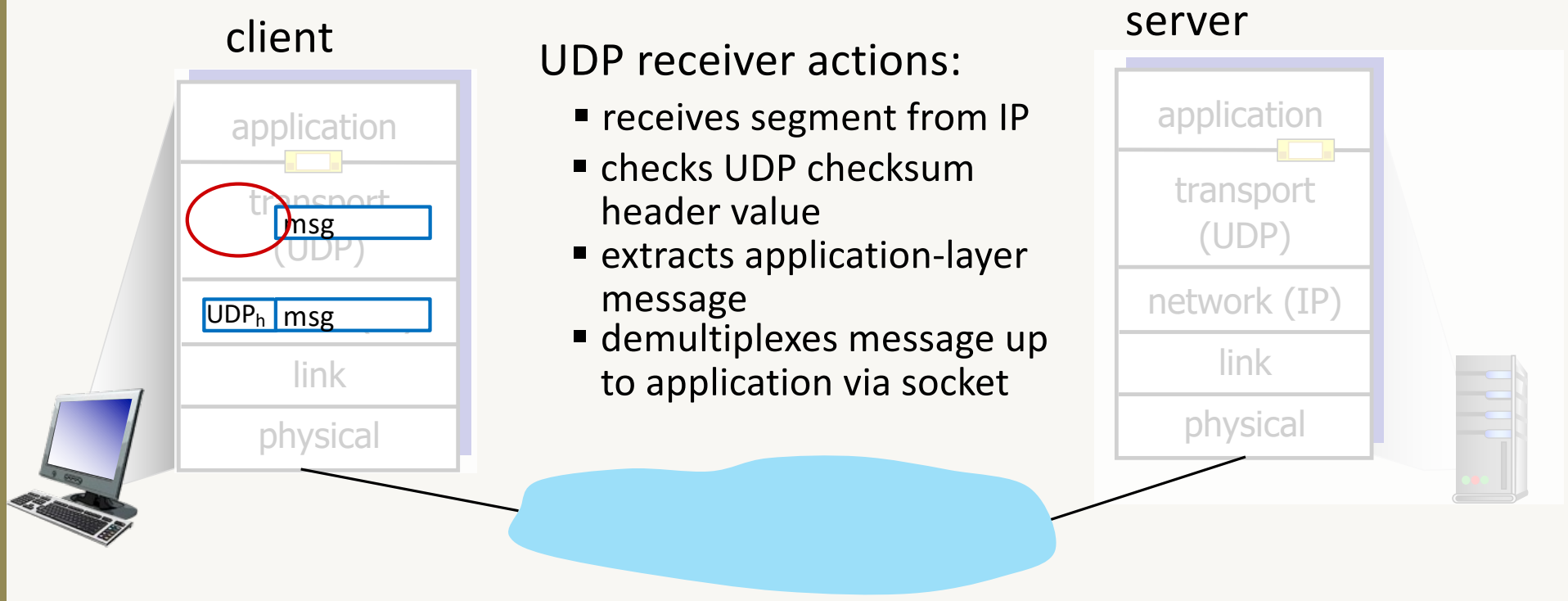


UDP sender actions:

- is passed an application-layer message
- determines UDP segment header fields values
- creates UDP segment
- passes segment to IP

UDP: Transport Layer Actions



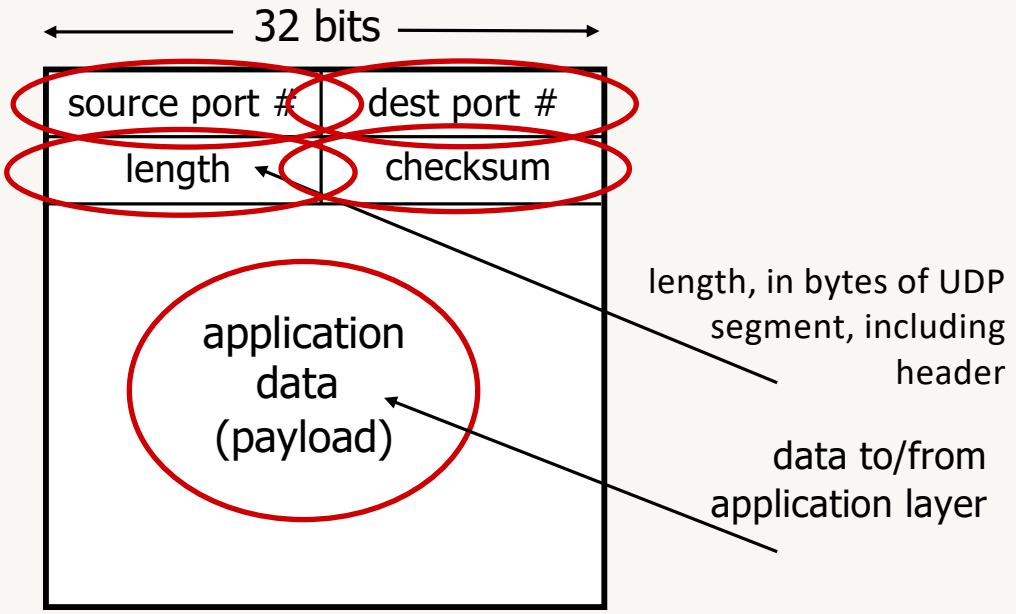


UDP receiver actions:

- receives segment from IP
- checks UDP checksum header value
- extracts application-layer message
- demultiplexes message up to application via socket

UDP Segment Header

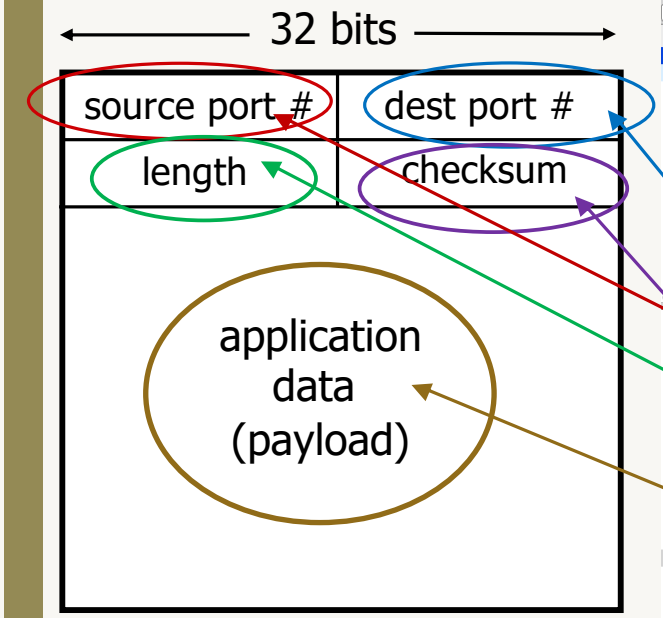




UDP segment format

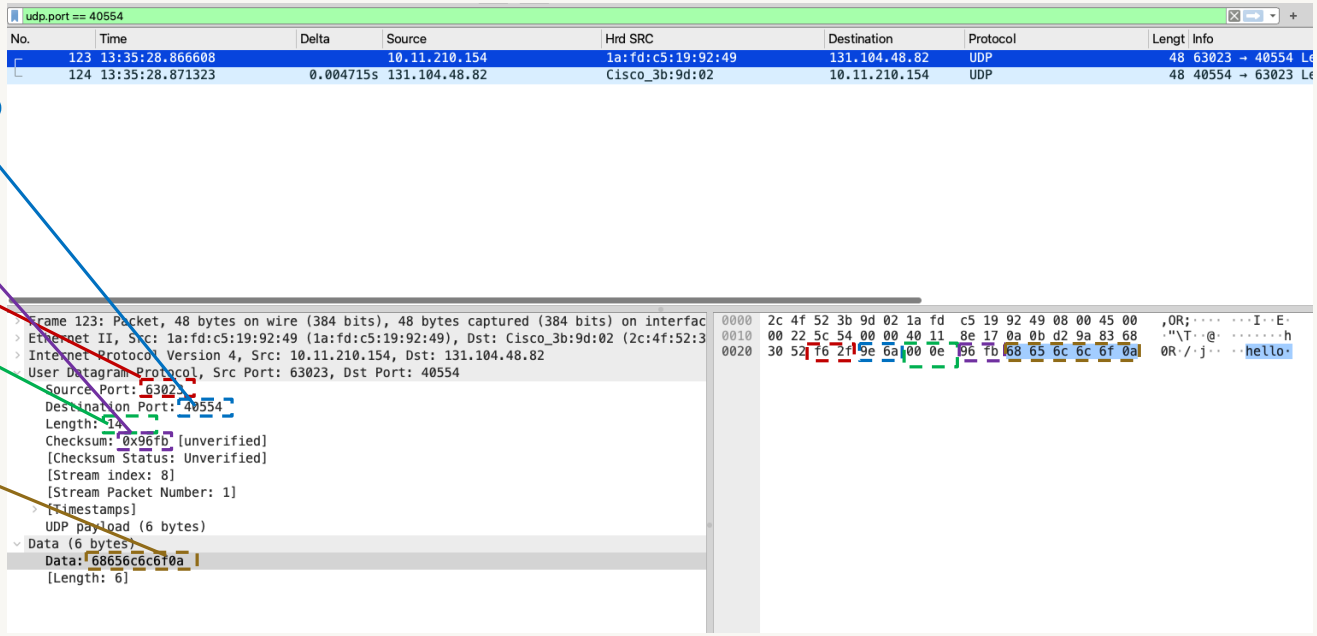
UDP Segment Header





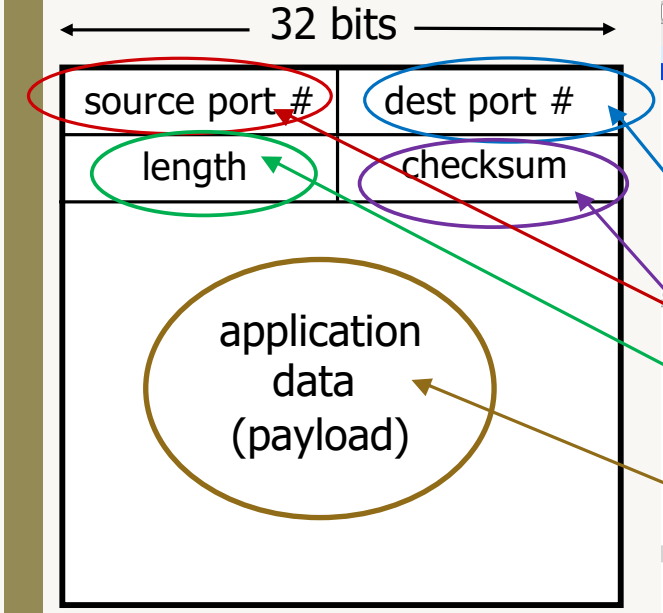
UDP segment format

Client's Request Packet



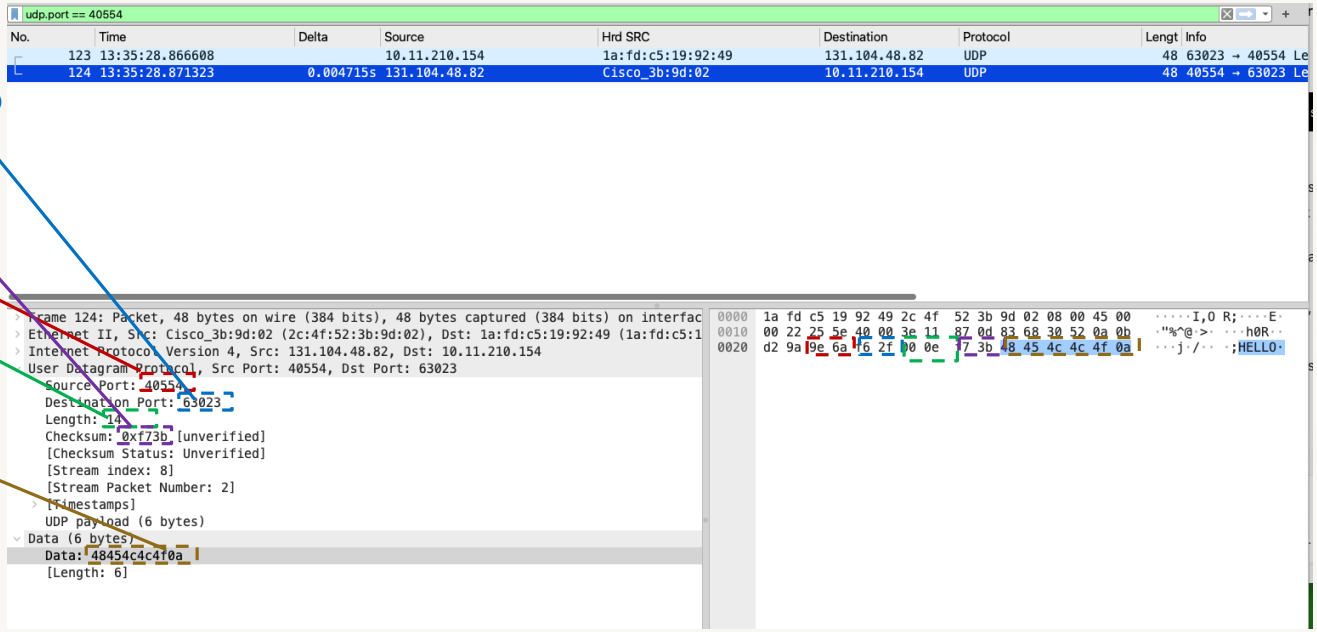
UDP Segment Header





UDP segment format

Server's Reply Packet





Discussion Question:

A request has UDP source port 50501 and destination port 50502. The server replies from the same socket to the client's sending socket. What source and destination ports should the reply use?

The reply uses source port 50502 and destination port 50501. The server's port becomes the reply's source port, and the client's original source port becomes the reply's destination port.

UDP Checksum



How checksum is calculated?

- Sender Side:
  - Segment contents are treated as 16-bit words
  - Perform one's complement sum (wrap around overflow)
  - Final result (one's complement) stored in the UDP checksum field
- Receiver Side
  - Receiver adds all 16-bit words, including the checksum
  - If the sum equals all 1s (binary), no error detected
  - If not, an error occurred during transmission

UDP Checksum



Example

Data: 0110011001100000 0101010101010101 1000111100001100

Sum of the first two words:

0110011001100000
0101010101010101

1011101110110101

Adding the third word:

1011101110110101
1000111100001100
1 0100101011000001

wraparound

Overflow is wrapped around

0100101011000010

1st complement of the sum

1011010100111101

This is checksum calculated by sender

At the receiver, all four 16-bit words are added, including the checksum:

- If no errors are introduced into the packet, then the sum will be 1111111111111111
- If one of the bits is a 0, then we know that errors have been introduced into the packet

UDP Checksum



At receiver side:

The checksum will be re-calculated for the whole received UDP segment including the checksum

Data: 0110011001100000 0101010101010101 1000111100001100

Checksum: 1011010100111101

From the previous slide, sum of the three words is: 0100101011000010

Adding the sender checksum:

$$\begin{array}{r} 0100101011000010 \\ 1011010100111101 \\ \hline 1111111111111111 \end{array}$$

UDP Checksum



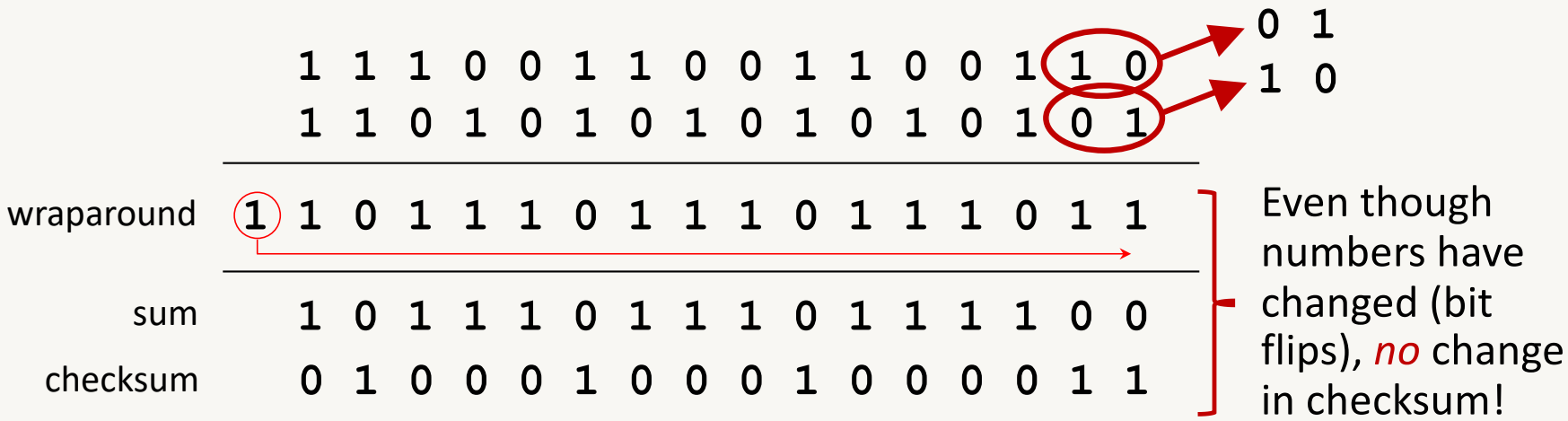
- **Why checksum is needed despite link-layer checks**
  - Not all links guarantee error checking
  - Errors can occur in router memory even if link transfer is correct
  - End-to-end provides error detection across the full path
- **Limitations of UDP checksum**
  - Only detects errors; does not correct or recover from them
  - UDP silently discards datagrams that fail checksum validation

UDP Checksum



Weak Protection:

example: add two 16-bit integers





Discussion Question:

A valid UDP segment arrives from IP. Describe how UDP turns it into data that the receiving application can use.

UDP checks the checksum, extracts the application-layer message, and demultiplexes the message to the application through the appropriate socket. The application receives its message rather than having to process the UDP header itself.



Discussion Question:

What information must an application provide to send a UDP message to another process?

It provides the destination IP address and destination port number. The IP address identifies the receiving host, and the port number identifies the receiving application's UDP socket.



Discussion Question:

Are the destination IP address and port number both stored in the UDP header?

No. The destination port number is stored in the UDP header. The destination IP address is stored in the IP header.



Discussion Question:

If the UDP header does not contain the destination IP address, how does IP know where to send the datagram?

The application supplies the destination IP address when it sends the message. The operating system passes this address to IP as additional information alongside the UDP datagram. IP uses it to fill in the destination address field.



Discussion Question:

Does UDP divide a large application message into several UDP datagrams?

No. UDP places each accepted application message in one UDP datagram. If the message exceeds the allowed UDP/IP size, the send operation fails, and the application must divide it into smaller messages.



Discussion Question:

A message fits within the UDP/IP size limit, but the resulting IP datagram exceeds the network MTU. What can happen?

IPv4 may fragment the IP datagram if fragmentation is allowed. This happens at the IP layer. UDP still treats the application message as one message.



Discussion Question:

If an IP datagram carrying a UDP message is fragmented, does the receiving application receive each fragment separately?

No. IP reassembles the fragments at the destination before passing the complete UDP datagram to UDP. The application receives one message, assuming successful delivery and a sufficiently large receive buffer.



Discussion Question:

An application divides a large message into three smaller messages and sends each using UDP. Will UDP automatically combine them at the receiver?

No. They are sent as three separate UDP datagrams. The receiving application must combine them if needed and handle any missing or out-of-order messages.

References



- Ch. 3, Kurose and Ross
- Ch. 24, Comer



Questions?